

Guía docente

Colecciones de datos I: Listas

Curso: Herramientas Computacionales

1. Información general

Tema	Colecciones de datos II: listas, indexación, slicing, métodos principales y recorridos.
Duración	2 horas (120 minutos).
Prerrequisitos	Variables, condicionales, ciclos <code>while/for</code> y colecciones estudiadas en la clase anterior.
Material principal	Notebook <code>Colecciones_Datos_II_Listas.ipynb</code> .
Enfoque	Concepto → ejemplo → predicción → ejecución → aplicación.

2. Propósito pedagógico

Esta segunda clase sobre colecciones debe aprovechar lo aprendido en la sesión anterior. La lista es probablemente la estructura que los estudiantes utilizarán con mayor frecuencia durante las siguientes semanas, por lo que es importante trabajarla con suficiente profundidad.

El objetivo no es memorizar una lista de métodos, sino comprender cuatro ideas:

1. una lista es una secuencia ordenada y modificable;
2. los índices permiten acceder a posiciones;
3. el slicing permite trabajar con partes de una secuencia;
4. los ciclos permiten procesar sistemáticamente sus elementos.

La pregunta transversal debe ser:

¿Qué necesito hacer con los datos y qué operación de lista me ayuda a hacerlo?

3. Objetivos de aprendizaje

Al finalizar la sesión, el estudiante debería poder:

1. Crear listas.
2. Consultar elementos mediante índices positivos y negativos.
3. Modificar elementos.
4. Utilizar slicing.
5. Explicar la diferencia entre `append()`, `insert()` y `extend()`.
6. Utilizar `remove()` y `pop()` correctamente.
7. Utilizar `index()` y `count()`.
8. Ordenar e invertir listas.

9. Recorrer listas con `for` y `while`.
10. Combinar listas con condicionales, acumuladores y contadores.
11. Elegir una lista frente a otras estructuras cuando se necesita una secuencia ordenada y modificable.

4. Distribución de los 120 minutos

Momento	Tiempo	Propósito
Introducción	10 min	Conectar con tuplas, conjuntos y diccionarios.
Creación e indexación	20 min	Crear listas y acceder por posición.
Slicing	20 min	Extraer partes de una lista.
Métodos principales	30 min	Modificar, consultar y ordenar listas.
Recorridos y condicionales	20 min	Procesar elementos.
Mini-reto	15 min	Aplicar los conceptos en un caso.
Cierre	5 min	Comprobación conceptual.

5. Estrategia didáctica

Se recomienda comenzar recordando las estructuras de la clase anterior:

Estructura	Característica	Uso típico
Tupla	Ordenada e inmutable	Datos agrupados que no deben cambiar.
Conjunto	Elementos únicos	Pertenencia y comparación de grupos.
Diccionario	Clave-valor	Asociar información.
Lista	Ordenada y modificable	Secuencias que debemos procesar o modificar.

La lista debe introducirse como una estructura que conserva muchas de las ventajas de una secuencia, pero permite modificarla.

6. Guion docente detallado

6.1 Introducción — 10 minutos

Comenzar con una pregunta:

Si tengo una colección de nombres que cambia durante el programa, ¿qué estructura vista hasta ahora podría utilizar?

Recordar que una tupla es inmutable.

Mostrar:

```
nombres = ("Ana", "Luis", "Maria")
```

Preguntar:

¿Qué ocurre si ahora quiero agregar a Sofia?

La respuesta es que no podemos modificar directamente la tupla.

Introducir entonces:

```
nombres = ["Ana", "Luis", "Maria"]
```

Explicar:

Una lista conserva el orden, tiene índices y, a diferencia de una tupla, puede modificarse.

6.2 Creación e indexación — 20 minutos

Mostrar:

```
estudiantes = ["Ana", "Luis", "Maria", "Carlos"]
```

Explicar:

- Los corchetes indican que estamos creando una lista.
- Los elementos están separados por comas.
- El orden se conserva.
- Los elementos pueden repetirse.

Construir en el tablero:

índice	0	1	2	3
valor	<i>Ana</i>	<i>Luis</i>	<i>Maria</i>	<i>Carlos</i>

Después mostrar índices negativos:

índice	-4	-3	-2	-1
valor	<i>Ana</i>	<i>Luis</i>	<i>Maria</i>	<i>Carlos</i>

Pregunta frecuente: “¿Por qué Python comienza en cero?”

Explicar que los índices se interpretan como desplazamientos desde el inicio.

Error de índice

Mostrar:

```
print(estudiantes[10])
```

Explicar el `IndexError` y relacionarlo con:

$$0 \leq \text{índice} < \text{len}(\text{lista})$$

Esta desigualdad es especialmente útil para conectar con los ciclos y `range()`.

Modificar elementos

Mostrar:

```
estudiantes[1] = "Pedro"
```

Comparar explícitamente con la tupla:

```
punto = (3, 5)
# punto[0] = 10
```

La pregunta importante es:

¿Qué propiedad de la lista permite esta operación?

Respuesta: la lista es mutable.

6.3 Slicing — 20 minutos

Introducir la sintaxis:

```
lista[inicio:fin]
```

Usar:

```
numeros = [10, 20, 30, 40, 50]  
print(numeros[1:4])
```

Explicar cuidadosamente:

- inicio = 1, sí se incluye;
- fin = 4, no se incluye;
- por tanto se obtienen los índices 1, 2 y 3.

Pregunta frecuente: “¿Por qué el último índice no se incluye?”

Respuesta pedagógica: esta convención facilita calcular la cantidad de elementos del intervalo como:

$$\text{fin} - \text{inicio}.$$

Por ejemplo, [1:4] contiene tres posiciones: 1, 2 y 3.

Inicio y fin omitidos

Mostrar:

```
numeros[:3]  
numeros[2:]  
numeros[:]
```

Explicar qué significa cada caso.

Paso

Mostrar:

```
numeros[: :2]  
numeros[1: :2]  
numeros[: : -1]
```

Punto importante: [: : -1] no es un método; es slicing con un paso negativo.

Preguntar antes de ejecutar:

¿Qué creen que hace `numeros[: :2]`?

Pedir que hagan una predicción.

6.4 Métodos principales — 30 minutos

Es importante no convertir esta sección en una memorización mecánica. Para cada método se recomienda plantear una necesidad.

`append()`

Pregunta:

Tengo una lista y quiero agregar un elemento al final. ¿Qué necesito?

Código:

```
frutas = ["manzana", "pera"]
frutas.append("uva")
```

Explicar que `append()` recibe un elemento y lo coloca al final.

`insert()`

Necesidad:

Quiero agregar un elemento en una posición específica.

Código:

```
frutas.insert(1, "mango")
```

El primer argumento es la posición; el segundo, el valor.

`extend()`

Mostrar:

```
a = [1, 2]
b = [3, 4]

a.extend(b)
```

Resultado:

```
[1, 2, 3, 4]
```

Después mostrar:

```
a = [1, 2]
a.append([3, 4])
```

Resultado:

```
[1, 2, [3, 4]]
```

Este contraste es especialmente importante.

`remove()` frente a `pop()`

Presentar la diferencia como una pregunta:

¿Quiero eliminar por valor o por posición?

- `remove(valor)` → busca por valor.
- `pop(indice)` → elimina por índice.

Además:

- `remove()` no devuelve el elemento eliminado.
- `pop()` sí devuelve el elemento eliminado.

Ejemplo:

```
numeros = [10, 20, 30]

elemento = numeros.pop(1)
print(elemento)
```

El resultado es 20.

`index()` y `count()`

Mostrar:

```
numeros = [10, 20, 30, 20, 40, 20]

print(numeros.index(20))
print(numeros.count(20))
```

Explicar:

- `index()` devuelve la primera posición.
- `count()` devuelve cuántas apariciones existen.

`sort()` y `reverse()`

Mostrar:

```
numeros.sort()
numeros.sort(reverse=True)
numeros.reverse()
```

Pregunta frecuente: “¿`reverse()` ordena de mayor a menor?”

Respuesta: no. Invierte la secuencia actual.

Pregunta frecuente: “¿`sort()` crea otra lista?”

Respuesta: el método modifica la lista sobre la que se ejecuta.

6.5 Recorridos — 20 minutos

Comenzar con el recorrido directo:

```
for estudiante in estudiantes:
    print(estudiante)
```

Explicar que normalmente esta es la forma más sencilla y legible de recorrer una lista cuando no necesitamos el índice.

Luego mostrar:

```
for i in range(len(estudiantes)):
    print(i, estudiantes[i])
```

Explicar por qué aquí sí necesitamos índices.

Finalmente conectar con `while`:

```
i = 0

while i < len(estudiantes):
    print(estudiantes[i])
    i += 1
```

Advertencia: recordar que olvidar `i += 1` puede producir un ciclo infinito.

6.6 Listas + condicionales — 20 minutos

Utilizar:

```
notas = [4.5, 2.8, 3.7, 4.9, 2.5]

for nota in notas:
    if nota >= 3.0:
        print(nota, "-> Aprobo")
    else:
        print(nota, "-> No aprobo")
```

Preguntar:

¿Qué está haciendo el ciclo y qué está haciendo el condicional?

Respuesta:

- El ciclo procesa cada nota.
- El condicional clasifica la nota.

Acumuladores y contadores

Introducir:

```
aprobados = 0

for nota in notas:
    if nota >= 3.0:
        aprobados += 1
```

Explicar la diferencia entre:

- **contador:** normalmente aumenta en una unidad;
- **acumulador:** suele sumar cantidades variables.

Ejemplo de acumulador:

```
suma = 0

for nota in notas:
    suma += nota
```

6.7 Mini-reto — 15 minutos

Plantear el inventario:

```
productos = ["pan", "leche", "arroz"]
```

Solicitar primero que los estudiantes indiquen qué método utilizarían para cada operación.

Necesidad	Método
Agregar al final	<code>append()</code>
Agregar en una posición	<code>insert()</code>
Eliminar por valor	<code>remove()</code>
Recorrer	<code>for</code>
Consultar pertenencia	<code>in</code>

La solución del notebook permite además practicar `input()` y el operador `in`.

7. Descripción paso a paso de los códigos principales

7.1 Indexación

```
estudiantes = ["Ana", "Luis", "Maria", "Carlos"]  
  
print(estudiantes[0])  
print(estudiantes[-1])
```

1. Se crea la lista.
2. `estudiantes[0]` solicita el primer elemento.
3. `estudiantes[-1]` solicita el último elemento.
4. `print()` muestra ambos valores.

7.2 Slicing

```
numeros = [10, 20, 30, 40, 50]  
parte = numeros[1:4]
```

1. Se crea la lista.
2. Se solicita desde el índice 1.
3. Se detiene antes del índice 4.
4. Se obtiene una nueva secuencia con los elementos de índices 1, 2 y 3.

7.3 `append()`

```
frutas = ["manzana", "pera"]  
frutas.append("uva")
```

Antes:

```
["manzana", "pera"]
```

Después:

```
["manzana", "pera", "uva"]
```

7.4 `extend()`

```
a = [1, 2]  
b = [3, 4]  
a.extend(b)
```

El contenido de `b` se incorpora elemento por elemento.

Resultado:

```
[1, 2, 3, 4]
```

7.5 Recorrido con índice

```
for i in range(len(estudiantes)):
    print(i, estudiantes[i])
```

1. `len(estudiantes)` obtiene la cantidad de elementos.
2. `range(...)` genera los índices válidos.
3. `i` recibe cada índice.
4. `estudiantes[i]` obtiene el elemento ubicado en esa posición.

7.6 Análisis de notas

El código central es:

```
aprobados = 0
no_aprobados = 0
suma = 0

for nota in notas:
    suma += nota

    if nota >= 3.0:
        aprobados += 1
    else:
        no_aprobados += 1
```

Explicar cada variable:

- `aprobados`: contador.
- `no_aprobados`: contador.
- `suma`: acumulador.
- `nota`: elemento actual de la lista.

En cada iteración:

$$\text{suma nueva} = \text{suma anterior} + \text{nota actual}$$

y además se clasifica la nota.

8. Preguntas frecuentes de los estudiantes

Pregunta 1: “¿Cuál es la diferencia entre lista y tupla?”

Ambas son secuencias ordenadas y permiten índices, pero la lista es mutable y la tupla es inmutable.

Pregunta 2: “¿Cuál es la diferencia entre lista y conjunto?”

Una lista conserva el orden y permite duplicados. Un conjunto está orientado a representar elementos únicos y no se consulta mediante índices.

Pregunta 3: “¿Puedo tener diferentes tipos de datos en una lista?”

Sí:

```
datos = ["Ana", 20, 4.5, True]
```

Sin embargo, debe explicarse que que Python lo permita no significa que mezclar tipos siempre sea una buena decisión de diseño.

Pregunta 4: “¿append() y insert() hacen lo mismo?”

Ambos agregan elementos, pero:

- `append()` agrega al final;
- `insert()` permite especificar una posición.

Pregunta 5: “¿Por qué `append([3,4])` y `extend([3,4])` son diferentes?”

Porque `append()` recibe un solo elemento. En este caso, ese elemento es la lista `[3,4]`.

`extend()` recorre la colección recibida y agrega sus elementos.

Pregunta 6: “¿`remove()` elimina todas las apariciones?”

No. Elimina la primera aparición del valor especificado.

Pregunta 7: “¿`pop()` necesita siempre un índice?”

No. Si se llama sin argumentos, elimina el último elemento.

Pregunta 8: “¿Qué diferencia hay entre `reverse()` y `sort(reverse=True)`?”

`reverse()` invierte el orden actual. `sort(reverse=True)` ordena los elementos de acuerdo con el criterio de ordenación, de forma descendente.

Pregunta 9: “¿Puedo recorrer una lista sin `range()`?”

Sí, y generalmente es la opción más sencilla cuando no necesitamos el índice:

```
for elemento in lista:  
    print(elemento)
```

Pregunta 10: “¿Cuándo necesito `range(len(lista))`?”

Cuando necesitamos trabajar explícitamente con las posiciones o necesitamos acceder simultáneamente a los índices y elementos mediante indexación.

9. Errores frecuentes que debe vigilar el docente

1. Confundir índice con valor.
2. Olvidar que el primer índice es cero.
3. Usar un índice fuera de rango.
4. Confundir slicing con indexación.
5. Pensar que el límite final del slicing se incluye.
6. Confundir `append()` con `extend()`.

7. Confundir `remove()` con `pop()`.
8. Confundir `reverse()` con ordenamiento.
9. Olvidar actualizar el índice en un `while`.
10. Intentar utilizar una lista como si fuera un conjunto o diccionario.

10. Actividad de comprobación rápida

Antes del cierre, plantear los siguientes fragmentos sin ejecutarlos y pedir predicciones:

Caso A

```
numeros = [10, 20, 30, 40]
print(numeros[1:3])
```

Respuesta:

```
[20, 30]
```

Caso B

```
lista = [1, 2]
lista.append([3, 4])
print(lista)
```

Respuesta:

```
[1, 2, [3, 4]]
```

Caso C

```
lista = [1, 2, 3]
elemento = lista.pop(1)
print(elemento)
print(lista)
```

Respuesta:

```
2
[1, 3]
```

Caso D

```
lista = [3, 1, 2]
lista.reverse()
print(lista)
```

Respuesta:

```
[2, 1, 3]
```

11. Orientación para los ejercicios de casa

Los ejercicios siguen una progresión deliberada:

- Ejercicio 1: creación e indexación.
- Ejercicio 2: slicing.
- Ejercicio 3: métodos.
- Ejercicio 4: búsqueda y conteo.
- Ejercicio 5: ciclos, contadores, acumuladores y análisis.
- Ejercicio 6: interacción con el usuario.
- Ejercicio 7: clasificación de datos.
- Ejercicio 8: integración de todos los conceptos.

El ejercicio 5 es especialmente importante porque prepara al estudiante para algoritmos iterativos de análisis de datos.

El ejercicio 8 puede funcionar como preparación para ejercicios posteriores donde se procesen datos reales.

12. Cierre conceptual

Finalizar con:

Una lista es una secuencia ordenada y modificable. Podemos acceder a sus elementos por posición, extraer partes mediante slicing, modificarla con métodos y recorrerla para procesar sus datos.

Las cuatro ideas que deben recordar:

1. **Índice:** “¿en qué posición está?”
2. **Slicing:** “¿qué parte de la secuencia necesito?”
3. **Método:** “¿qué operación quiero realizar sobre la lista?”
4. **Recorrido:** “¿cómo proceso todos sus elementos?”

La clase siguiente puede partir de esta base para introducir procesamiento más sistemático de colecciones, funciones sobre datos o estructuras anidadas, dependiendo de la secuencia curricular.